SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Pattern Recognition and Text Processing

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Pattern Recognition and Text Processing
Weightage:	40% of CIA		
CO5 Statement:	Analyse regular expression patterns. (BL-3)		
Student Name:	N.Somashekar Naidu	**Reg. No.:**	192472347
Section / Batch:	2024	**Date:**	10th July 2026

Code of Conduct

I N.Somashekar Naidu*(192472347)Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _N.Somashekar Naidu_____

Instructions

- Show all intermediate steps, calculations, state transitions, and reasoning wherever applicable.
- Use only the Python re (Regular Expressions) module to solve the given problems.
- Clearly mention the Regular Expression (Regex) pattern used before writing the corresponding Python program.
- Display the required output for each question, including extracted values, validation results, counts, and positions wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Generation	Pattern Matching Information Extraction	1
Search for a String	Keyword Search and Text Analysis	2
Split the documentation into sentences and words	Text Tokenization and Sentence Segmentation	3
Email and Strong Password Validation	Data Validation and Format Verification	4
Compile Once, Validate Many	Pattern Reusability and Performance Optimization	5

Annexure A – Pattern Recognition and Text Processing

Section 1: Regular Expression Pattern Generation

Student Details:
Name: John Doe
Email: john.doe123@gmail.com
Mobile: +91-9876543210
Password: P@ssw0rd123
Date of Birth: 15/08/2004
Register Number: 23AIML1056
Department: Artificial Intelligence and Machine Learning
For the given student details formulate Regex patterns for the following:
Email ID
Mobile Number
Strong Password
Date of Birth (DD/MM/YYYY)
Register Number

Section 2: Search for a String

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.
Write a Python program using the re module to perform the following operations for the word "AI":

1. Find the first occurrence of the word "AI" using re.search().
2. Display the starting position of the first occurrence.
3. Display the ending position of the first occurrence.
4. Display the total number of times the word "AI" appears in the passage.
5. Print an appropriate message if the word is not found.

Section 3: Split the documentation into sentences and words

Refer the previous passage and Write a Python program using the re.split() method to perform the following operations:

- Split the passage into sentences using appropriate punctuation (., ?, !) as delimiters.
- Count and display the total number of sentences.
- Split the passage into words by considering one or more whitespace characters as delimiters.
- Count and display the total number of words.
- Display the list of extracted sentences and words.

Section 4: Email and Strong Password Validation

A website requires users to register by providing an Email ID and a Strong Password. Write a Python program using the re module to validate the following inputs.
Validation Criteria:
Email ID

- Must begin with one or more letters or digits.
- May contain ., _, %, +, or - before the @ symbol.
- Must contain a valid domain name.
- Must end with a valid domain extension such as .com, .org, .edu, or .in.

Strong Password

- Must contain at least 8 characters.
- Must contain at least one uppercase letter.
- Must contain at least one lowercase letter.
- Must contain at least one digit.
- Must contain at least one special character from @, #, $, %, &, !, or *.
- Must not contain any whitespace characters.

Section 5: Compile Once, Validate Many

A company receives 1,000 email IDs from its customers and needs to validate each one efficiently. Instead of compiling the same regular expression for every email, write a Python program using the re.compile() method to compile the pattern once and reuse it to validate all the email IDs.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q. No. / Criteria	Conceptual Understanding	Accuracy of Answer	Application of Knowledge	Completeness of Response	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SECTION - 1

1. Email ID Validation
Email:
joshithagoud15@gmail.com
Regex Pattern
^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$
Python Program

```
import re
email = "penumadhunikhita@gmail.com"
pattern = r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
match = re.fullmatch(pattern, email)
print("Email ID:", email)
print("Regex Pattern:", pattern)
if match:
    print("Validation Result: Valid Email ID")
    print("Matched Value:", match.group())
    print("Starting Position:", match.start())
    print("Ending Position:", match.end() - 1)
else:
    print("Validation Result: Invalid Email ID")
```

Expected Output
Email ID: penumadhunikhita@gmail.com
Regex Pattern: ^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$

Validation Result: Valid Email ID
Matched Value: penumadhunikhita@gmail.com
Starting Position: 0
Ending Position: 28
2. Mobile Number Validation
Mobile Number
9059936930
Regex Pattern Used
^[6-9]\d{9}$
Python Program

```
import re
mobile = "9059936930"
pattern = r'^[6-9]\d{9}$'
match = re.fullmatch(pattern, mobile)
print("Mobile Number:", mobile)
print("Regex Pattern:", pattern)
if match:
    print("Validation Result: Valid Mobile Number")
    print("Matched Value:", match.group())
    print("Starting Position:", match.start())
    print("Ending Position:", match.end() - 1)
else:
    print("Validation Result: Invalid Mobile Number")
```

Expected Output
Mobile Number: 9059936930
Regex Pattern: ^[6-9]\d{9}$
Validation Result: Valid Mobile Number
Matched Value: 9059936930
Starting Position: 0
Ending Position: 9

3. Strong Password Validation

Password

1w2e$%345678!!&^

Regex Pattern Used

^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#$%^&+=!]).{8,}$

Python Program

```
import re
password = "1w2e$%345678!!&^"
pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#$%^&+=!]).{8,}$'
match = re.fullmatch(pattern, password)
print("Password:", password)
print("Regex Pattern:", pattern)
if match:
    print("Validation Result: Strong Password")
    print("Matched Value:", match.group())
    print("Starting Position:", match.start())
    print("Ending Position:", match.end() - 1)
else:
    print("Validation Result: Weak Password")Expected Output
```

Strong Password

Output

```
Password: 1w2e$%345678!!&^
Regex Pattern: ^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#$%^&+=!]).{8,}$
Validation Result: Weak Password
```

4. Date of Birth Validation (DD/MM/YYYY)

Date of Birth

21-11-1006

Regex Pattern Used

^(0[1-9]|[12][0-9]|3[01])-(0[1-9]|1[0-2])-\d{4}$

Python Program

```
import re
dob = "21-11-1006"
pattern = r'^(0[1-9]|[12][0-9]|3[01])-(0[1-9]|1[0-2])-\d{4}$'
match = re.fullmatch(pattern, dob)
print("Date of Birth:", dob)
print("Regex Pattern:", pattern)
if match:
    print("Validation Result: Valid Date of Birth")
    print("Matched Value:", match.group())
    print("Starting Position:", match.start())
    print("Ending Position:", match.end() - 1)
else:
    print("Validation Result: Invalid Date of Birth")**Expected Output**
```

Valid Date of Birth

Output

```
Date of Birth: 21-11-1006
Regex Pattern: ^(0[1-9]|[12][0-9]|3[01])-(0[1-9]|1[0-2])-\d{4}$

Validation Result: Valid Date of Birth
Matched Value: 21-11-1006
Starting Position: 0
Ending Position: 9
```

5. Register Number Validation
Register Number
192425353
Regex Pattern Used
^\d{9}$
Python Program
```
import re
register_no = "192425353"
pattern = r'^\d{9}$'
match = re.fullmatch(pattern, register_no)
print("Register Number:", register_no)
print("Regex Pattern:", pattern)
if match:
    print("Validation Result: Valid Register Number")
    print("Matched Value:", match.group())
    print("Starting Position:", match.start())
    print("Ending Position:", match.end() - 1)
else:
    print("Validation Result: Invalid Register Number")
```
Output
Register Number: 192425353
Regex Pattern: ^\d{9}$
Validation Result: Valid Register Number
Matched Value: 192425353
Starting Position: 0
Ending Position: 8

SECTION – 2

Finding the Word "AI" Using Python re Module
Text Passage
Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.

Regex Pattern Used
\bAI\b
Pattern Explanation
- \b → Matches a word boundary.
- AI → Matches the exact word **AI**.
- \b → Ensures only the whole word "AI" is matched.

Python Program (Using Only re Module)
```
import re

text = """Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand."""
pattern = r'\bAI\b'
match = re.search(pattern, text)
if match:
    print("First Occurrence of 'AI':", match.group())
    print("Starting Position:", match.start())
```

```
    print("Ending Position:", match.end() - 1)
    occurrences = re.findall(pattern, text)
    print("Total Number of Occurrences:", len(occurrences))
else:
    print("The word 'AI' was not found.")
```

Expected Output

First Occurrence of 'AI': AI
Starting Position: 25
Ending Position: 26
Total Number of Occurrences: 6

SECTION – 3

Splitting the Passage into Sentences and Words Using Python re.split()

Text Passage

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.

Regex Patterns Used

1. Split into Sentences

[.!?]+\s*

Pattern Explanation

- [.!?]+ → Matches one or more sentence-ending punctuation marks (., !, ?).
- \s* → Matches zero or more whitespace characters after the punctuation.

2. Split into Words

\s+

Pattern Explanation

- \s+ → Matches one or more whitespace characters (space, tab, newline).

Python Program (Using Only re Module)

```
import re
text = """Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand."""
sentence_pattern = r'[.!?]+\s*'
word_pattern = r'\s+'
sentences = re.split(sentence_pattern, text)
sentences = [sentence for sentence in sentences if sentence]
words = re.split(word_pattern, text)
words = [word for word in words if word]
print("Extracted Sentences:")
for i, sentence in enumerate(sentences, 1):
    print(f"{i}. {sentence}")
print("\nTotal Number of Sentences:", len(sentences))
print("\nExtracted Words:")
print(words)
print("\nTotal Number of Words:", len(words))
```

Expected Output

Extracted Sentences:
1. Artificial Intelligence (AI) is transforming industries across the world
2. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences
3. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-

making
4. As AI continues to evolve, professionals with AI skills are in high demand
Total Number of Sentences: 4
Extracted Words:
['Artificial', 'Intelligence', '(AI)', 'is', 'transforming', 'industries', 'across', 'the', 'world.', 'AI', 'is', 'used', 'in', 'healthcare', 'to', 'assist', 'doctors', 'in', 'diagnosis,', 'in', 'banking', 'to', 'detect', 'fraud,', 'and', 'in', 'education', 'to', 'provide', 'personalized', 'learning', 'experiences.', 'Many', 'companies', 'invest', 'heavily', 'in', 'AI', 'research', 'because', 'AI', 'improves', 'efficiency', 'and', 'enables', 'intelligent', 'decision-making.', 'As', 'AI', 'continues', 'to', 'evolve,', 'professionals', 'with', 'AI', 'skills', 'are', 'in', 'high', 'demand.']
Total Number of Words: 59

SECTION-4

Website Registration Validation Using Python re Module

Email ID

penumadhunikhita@gmail.com

Regex Pattern for Email ID

^[A-Za-z0-9][A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$

Pattern Explanation

- ^ → Beginning of the string.
- [A-Za-z0-9] → Email must start with a letter or digit.
- [A-Za-z0-9._%+-]* → Allows letters, digits, ., _, %, +, and - before @.
- @ → Matches the @ symbol.
- [A-Za-z0-9.-]+ → Matches the domain name.
- \. → Matches the dot before the domain extension.
- (com|org|edu|in) → Allows only .com, .org, .edu, or .in.
- $ → End of the string.

Strong Password

`Nikhita@123`

Regex Pattern for Strong Password

^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!*])[^\s]{8,}$

Pattern Explanation

- ^ → Beginning of the string.
- (?=.*[A-Z]) → At least one uppercase letter.
- (?=.*[a-z]) → At least one lowercase letter.
- (?=.*\d) → At least one digit.
- (?=.*[@#$%&!*]) → At least one special character (@, #, $, %, &, !, *).
- [^\s]{8,} → Minimum 8 characters with no whitespace.
- $ → End of the string.

Python Program (Using Only re Module)

```
import re
email = "penumadhunikhita@gmail.com"
password = "Nikhita@123"
email_pattern = r'^[A-Za-z0-9][A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!*])[^\s]{8,}$'
email_match = re.fullmatch(email_pattern, email)
password_match = re.fullmatch(password_pattern, password)
print("Email ID:", email)
print("Password:", password)
print("\nValidation Results")
if email_match:
    print("Email ID: Valid")
    print("Matched Email:", email_match.group())
else:
    print("Email ID: Invalid")
if password_match:
    print("Password: Strong")
    print("Matched Password:", password_match.group())
else:
```

```
    print("Password: Weak")
```

Expected Output

Email ID: penumadhunikhita@gmail.com
Password: Nikhita@123
Validation Results
Email ID: Valid
Matched Email: penumadhunikhita@gmail.com
Password: Strong
Matched Password: Nikhita@123

SECTION - 5

Email Validation Using re.compile() in Python

Regex Pattern Used

^[A-Za-z0-9][A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$

Pattern Explanation

- ^ → Beginning of the string.
- [A-Za-z0-9] → Email must start with a letter or digit.
- [A-Za-z0-9._%+-]* → Allows letters, digits, ., _, %, +, and - before the @ symbol.
- @ → Matches the @ symbol.
- [A-Za-z0-9.-]+ → Matches the domain name.
- \. → Matches the dot before the extension.
- (com|org|edu|in) → Allows only the domain extensions .com, .org, .edu, and .in.
- $ → End of the string.

Python Program (Using Only `re` Module)

```
import re
# List of email IDs
emails = [
    "penumadhunikhita@gmail.com",
    "john123@yahoo.com",
    "student_01@college.edu",
    "employee+hr@company.org",
    "user@website.in",
    "invalidemail.com",
    "@gmail.com",
    "user@gmail",
    "user name@gmail.com",
    "abc#gmail.com"
]

email_pattern = re.compile(
    r'^[A-Za-z0-9][A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$'
)
print("Email Validation Results\n")
valid_count = 0
invalid_count = 0

for email in emails:
    if email_pattern.fullmatch(email):
        print(email, "-> Valid")
        valid_count += 1
    else:
        print(email, "-> Invalid")
        invalid_count += 1
print("\nTotal Emails:", len(emails))
print("Valid Emails:", valid_count)
print("Invalid Emails:", invalid_count)
```

OUTPUT:

```
penumadhunikhita@gmail.com -> Valid
john123@yahoo.com -> Valid
student_01@college.edu -> Valid
employee+hr@company.org -> Valid
user@website.in -> Valid
```

invalidemail.com -> Invalid
@gmail.com -> Invalid
user@gmail -> Invalid
user name@gmail.com -> Invalid
abc#gmail.com -> Invalid
Total Emails: 10
Valid Emails: 5